CodeAgapay AI Learning Plan Framework

Version 2.1 · Accepted by the team · 9 November 2026
Printable snapshot of docs/plan-framework.md, the source of truth. Replaces the general-purpose “AI Learning Plan Framework and Resource Dictionary” (v1) for use inside CodeAgapay.

0. How to read this document

Version 1 described how an AI should build a learning plan for *any* topic. This version adapts it to CodeAgapay and to the team’s decision: **the AI creates the whole learning plan and its lessons, guided by this framework and a verified list of resources.** Every rule is marked so the team can see what is truly in the app, what the first implementation will add, and what waits for later.

Label	Meaning
Built	Already in the app today (as of commit 160cbf1). Reused by the new plan.
v1 plan	Part of the first AI-generated plan implementation proposed in this document.
Later	Agreed direction, but not in the first implementation.

Enforced by	Meaning
Prompt	The AI is told to follow the rule. Nothing checks it automatically, so the team reviews samples.
Code	The app checks or carries out the rule itself. It holds even when the AI ignores the prompt.
Team	People do it: curators, reviewers or whoever runs the regular jobs.

> **Key principle:** if a rule matters for safety, grading or progress, it is enforced in **Code**, not only in the prompt. Prompts guide the content; code guards the result.

1. Purpose and scope

- **What the AI does:** writes each learner’s plan outline (modules, lessons, project) at sign-up, and writes each lesson (explanation, example, practice, quiz) the first time the learner opens it.
- **What grounds it:** this framework (as short prompts), the learner’s answers and skill check results, the verified resource list in section 9, and any curated links the app already has for that subject.
- **Scope:** CodeAgapay’s skill paths only (Make a website, Automate boring tasks, Work with data, Make games and fun projects, Get ahead in CS class) and their tracks (Python, JavaScript, HTML, CSS, Java, SQL). Not any topic. The non-coding parts of v1 (spoken languages, soft skills, creative skills) are removed.
- **Honest limit of v1:** lesson explanations come from the AI model’s own knowledge, guided by the rules. The AI does not yet read the linked web pages and teach from them. That is “Retrieval” in section 13 (Later).

2. How a plan is generated

Two small AI steps instead of one very large one. Small steps fit the free AI tiers, are faster, and let each lesson adapt to how the learner is doing.

Step	When	What happens	Status
1. Onboarding	Sign-up	Path, project, experience, topics used, hours per week, optional date, what helps when stuck. Then the short skill check.	**Built**
2. Outline	After the learner confirms	One AI call writes the plan outline (section 5). Code checks it (section 10) and saves it. Weeks are assigned by code from hours per week.	**v1 plan**
3. Lesson	First time a lesson is opened	One AI call writes that lesson (section 6), using the learner's latest quiz results. Code checks it and saves it. Later visits show the saved lesson.	**v1 plan**
4. Check	End of each lesson	Code grades the mini-check and applies the score bands (section 7).	**v1 plan**
5. Adapt	After a low score	The next generated version of the lesson explains it differently (section 8). Retakes use new questions.	**v1 plan**
Fallback	AI off, over its limit, or replies badly twice	The current curated plan is built instead (curated topics, rule-based ordering and weeks, template project).	**Built**

> **Fallback gap:** curated topics exist only for Python today (8 topics; HTML and CSS are drafted). For other tracks the fallback can only show setup steps and a message to try again later.

3. Learner inputs

v1 listed seven inputs. This is how each maps to what CodeAgapay collects.

v1 input	In CodeAgapay	Status	Notes
Topic	Skill path, plus the learner's own project idea	**Built**	The path decides the tracks.
Goal	Goal text and project idea	**Built**	Decides what is core and what is optional.
Level	Experience answer plus skill check results per topic	**Built**	Mapped to beginner, intermediate or advanced for the depth table.
Time per session	Not asked	**v1 plan**	Default 30 minutes and state it as an assumption. Add a question later if needed.
Total time or deadline	Hours per week and optional target date	**Built**	The app already flags a plan that misses the date.
Prior knowledge	Skill check statuses: shown, partly there, to work on, not checked	**Built**	"Shown" topics become recaps or are skipped.
Preferred format	"What helps when stuck" (hint style)	**Built**	Used for hints today; v1 plan also passes it to lesson generation.

Missing inputs: assume a complete beginner with 30-minute sessions, and list every assumption at the top of the plan so the learner can correct it. *Enforced by Prompt; v1 plan.*

4. Core principles

Principle	Status	Enforced by	How
Beginner first. Assume no prior knowledge unless shown.	Built	Code + Prompt	Skill check evidence decides what is known; prompts write for beginners.
Build in layers. A lesson only uses ideas taught earlier.	v1 plan	Prompt + Code	Outline lists prerequisites per lesson; code rejects a lesson that needs a later one.
One new idea at a time.	v1 plan	Prompt	Depth table, section 6.
Learn by doing. Every lesson has a try-it task.	v1 plan	Code	A lesson without a practice task fails validation.
Check often, low stakes.	v1 plan	Code	Mini-check after every lesson.
Tie everything to the goal.	v1 plan	Prompt + Code	Lessons are tagged core or optional; optional ones are cut first.
Show the finish line.	v1 plan	Code	Every lesson needs a “You will be able to” line; the plan states its goal.
Never invent resources.	v1 plan	Code	Links outside the allowed sites are removed (section 9). Today: AI never writes links at all (Built).
Recover, do not repeat.	v1 plan	Code + Prompt	After a low score the lesson is regenerated with a different explanation and example.
Spaced recall.	Built	Code	Warm-ups from the previous topic; passed topics return for review after 7 days.
Visible progress.	Built	Code	Plan page leads with one next step; finish screen. Module progress lines are v1 plan.
Hints, never answers. (new)	Built	Prompt	The help panel gives three levels of hints in English or Taglish.

5. Plan outline (written at sign-up)

Every plan has these parts, in this order.

Part	What it holds	Status
Overview	Goal, assumed level, total lessons and hours, assumptions made, any conflict (for example, a big goal and a close date) with a realistic version.	v1 plan
Setup	The track’s curated setup steps, ending with its verification check (for example: run print("Hello!") and see Hello! printed). Not written by the AI.	Built
Foundations	The basic building blocks, as recaps when the skill check showed them.	v1 plan
Modules	3 to 5 lessons each, simple to complex, each with a one-sentence goal, a milestone (“you can now…”) and a stuck path.	v1 plan
Lessons (titles only)	Title, “You will be able to”, minutes, core or optional, prerequisites.	v1 plan
Project	One project tied to the goal, with weekly steps and “done when” checks. Adapted from a curated idea when one fits.	Built
Next steps and glossary	What to learn next; every key term from the saved lessons, in order, shown on the finish screen.	v1 plan

Sequencing rules

- Order lessons so each one only needs what came before. *Prompt + Code (prerequisite check). v1 plan.*
- Give a first small win within the first two lessons after setup. *Prompt. v1 plan.*
- No lesson much harder than the one before it. *Prompt. v1 plan.*
- Include only what the goal needs. *Prompt. v1 plan.*
- Spread lessons over weeks from hours per week; a lesson that does not fit starts the next week. *Code. Built (reused from today's planner).*
- When time is short, cut optional lessons first; never cut setup, key terms, mini-checks or the project. *Code. v1 plan.*

6. Lessons (written when first opened)

Every lesson has the same parts. Code rejects a lesson that is missing a required part.

Part	Rule	Status
You will be able to	One sentence.	**v1 plan**
Time	Within the depth table's range for the level.	**v1 plan**
Warm-up	1 to 2 ungraded questions on the previous lesson, preferring ones the learner missed; none in the first lesson. Built for curated topics (journey.warm_up); v1 plan for generated lessons.	**Built**
Key terms	Up to 3 for beginners (5 intermediate, 7 advanced), each with a plain definition. Today's curated topics already keep to 3.	**Built**
Why it matters	The problem this solves, before the how.	**Built**
Explanation	At the depth for the level. No internals or history for beginners.	**Built**
Example	A worked code example with its exact output. v2 asks for 2 at beginner level (one simple, one real-world, ideally from the learner's path).	**v1 plan**
Try it yourself	Copy an example, change it, then build something small. No files, internet or extra installs at beginner level.	**Built**
Resources	1 primary and up to 2 optional, from section 9 only.	**v1 plan**
Mini-check	2 to 3 auto-graded questions (1 if sessions are under 20 minutes).	**v1 plan**

> "Built" here means CodeAgapay's curated topics already follow the rule (and the drafting prompts enforce it). The v1 plan moves the same rules into the lesson-generation prompt.

Depth table

Setting	Beginner	Intermediate	Advanced
Main ideas per lesson	1	1 to 2	2 to 3
New key terms	Up to 3	Up to 5	Up to 7
Depth	What it is, why it matters, how to use it	Adds trade-offs and common mistakes	Adds how it works inside and edge cases
Examples per idea	2: one simple, one real-world	2 real-world	1 real-world, 1 edge case
Lesson length	15 to 30 minutes	25 to 45 minutes	30 to 60 minutes
Practice	Guided with hints	Partly guided	Open-ended

Change from today: curated topics run 45 to 120 minutes and the drafting prompt allows 30 to 120. Generated lessons follow the table above, so one curated topic may become two or three lessons.

Taglish and English

This is about the language the AI writes in, for the outline, lessons, hints and feedback. It is not about teaching spoken languages; that part of version 1 is removed.

Rule	Enforced by	Status
Match the learner: if they write in Taglish, reply in Taglish; otherwise plain English.	Prompt + Code	**Built**
Detect Taglish from common Tagalog words and add an explicit “reply in Taglish” note, because models drift back to English.	Code	**Built**
Plans are written in Taglish when the learner used Taglish.	Prompt	**Built**
Apply the same rule to generated lessons, using the learner’s onboarding and help messages.	Prompt + Code	**v1 plan**
Keep code, keywords, function names and error messages in English, even in Taglish text.	Prompt	**v1 plan**
Write Taglish the way Filipino students mix it, not formal Tagalog. Give the prompt 2 or 3 short sample explanations to copy.	Prompt	**v1 plan**
Tests check that a Taglish learner gets a Taglish lesson and an English learner gets English.	Code	**v1 plan**

7. Checks, quizzes and score bands

Question rules

- **Only formats the app can grade by itself:** multiple choice (with the correct choice stored) and fill-in-the-blank (with every accepted answer). *Code. v1 plan.* Fill-in-the-blank is Built today.
- “Explain” questions are shown as ungraded self-checks with a sample answer. *Code. v1 plan.* AI grading of written answers is Later.
- About 50% recall, 30% apply, 20% explain. *Prompt. v1 plan.*
- Every question has the correct answer and a one-sentence explanation. *Code. v1 plan.*
- Questions only test what has been taught. No trick questions for beginners. *Prompt. v1 plan.*

Checkpoints

Checkpoint	Where	Status
Setup check	End of setup, instead of a quiz.	Built
Mini-check	End of every lesson.	v1 plan
Topic quiz	Today's curated fill-in-the-blank quiz per topic (fallback plans).	Built
Module quiz	After every 3 lessons (beginner), 4 (intermediate), 5 (advanced); 5 to 8 questions with at least 2 from earlier modules.	Later
Final check	Right before the project; 10 to 15 questions across all modules.	Later
Project	End of the plan; "done when" checks.	Built

Score bands (what happens after a check)

Score	What the plan does	Status
Pass mark	70%, compared as exact fractions so 69.5% never passes.	Built
100%	Continue.	Built
70% to 99%	Continue. Missed questions come back first in the next warm-up.	Built
50% to 69%	Suggest another resource and retake. v1 plan: the retake has new questions.	Built
Below 50%	Re-teach. v1 plan: regenerate the lesson with a different explanation and example, and check no prerequisite was skipped.	Built
Two misses	Mark the topic shaky, continue the plan, bring it back as a review later.	Built
85% before the lesson	Skip the lesson but still show its key terms. (Today a curated topic can be skipped by passing its quiz at 70%.)	Later

A failed check never blocks the learner for good; there is always a next step. *Code. Built.*

8. Adapting to the learner

Signal	What changes	Status
Skill check result	"Shown" topics are skipped; "partly there" become recaps.	Built
Experience and results	Pick the depth table column.	v1 plan
Low score on a lesson	The lesson is regenerated: new explanation, new example, a different resource type.	v1 plan
Missed questions	Come back first in the next warm-up. Built for curated topics; v1 plan for generated lessons and retakes.	Built
What helps when stuck	Shapes hints today; v1 plan also shapes lesson examples and resource types.	Built
Pace change	Learner changes hours per week; weeks are recomputed, progress kept.	Built
Missed sessions	Progress never resets; opt-in reminder emails.	Built
Plans over 2 weeks	One catch-up session per week; a review lesson after every 2 modules.	Later

9. Resources and grounding

Rules

- **Allowed sites only.** A link is kept only if its site is on the allowed list below; otherwise the code drops the URL and keeps the resource name and a search phrase. *Code. v1 plan.*
- **Curated links first.** When the app already has checked links for the subject, they are given to the AI and preferred. *Prompt. v1 plan.*
- **Free first.** Paid resources are not on the list. *Team. v1 plan.*
- **Match the level.** No advanced documentation for beginners. *Prompt. v1 plan.*
- **Official sources for setup.** Setup stays curated. *Team. Built.*
- **Resources support the lesson;** they never replace the explanation inside it. *Prompt. v1 plan.*
- **Broken links:** the weekly link check marks broken curated links. *Team runs check_links. Built.* Checking AI-chosen links the same way is Later.

Allowed list (coding and free only)

Trimmed from v1's resource dictionary. Each row must be opened and checked by the team before launch, with the date it was checked.

Resource	Site	Best for	Level
MDN Web Docs	developer.mozilla.org	HTML, CSS, JavaScript	Beginner to advanced
Python docs	docs.python.org	Python tutorial and reference	Beginner to advanced
W3Schools	w3schools.com	Quick beginner examples	Beginner
freeCodeCamp	freecodecamp.org, youtube.com/@freecodecamp	Long beginner courses	Beginner to intermediate
The Odin Project	theodinproject.com	Web development path	Beginner to intermediate
Harvard CS50	cs50.harvard.edu	Intro to programming	Beginner
Khan Academy	khanacademy.org	Intro computing, JS drawing	Beginner
Kaggle Learn	kaggle.com/learn	Python, data analysis	Beginner to intermediate
Real Python (free articles)	realpython.com	Python articles	Beginner to advanced
Exercism	exercism.org	Practice exercises	Beginner to advanced
Google Colab	colab.research.google.com	Python with no install (phones, school PCs)	Beginner
CodePen	codepen.io	HTML, CSS, JS with no install	Beginner
Official installers	python.org, nodejs.org, code.visualstudio.com, git-scm.com	Setup only	Beginner
Stack Overflow	stackoverflow.com	Searching error messages	Beginner to advanced

Removed from v1: paid or mostly paid (Brilliant, Udemy, Google certificates, Codecademy Pro, LeetCode premium) and non-coding (Duolingo, Forvo, Easy Languages, language exchange apps, Kahoot, Investopedia, Desmos, Wolfram Alpha, OpenStax, Project Gutenberg, Open Library). YouTube channels

are allowed only by channel and search phrase until specific videos are checked.

10. Checks on every AI reply

These run in code before anything is saved or shown. They are what make AI-generated plans safe to show without a person approving each one first.

Check	If it fails	Status
The reply is valid JSON in the expected shape (appendix A).	Retry once, then fall back.	Built (JSON) / v1 plan (shape)
Required parts present (section 6); lesson minutes within the depth table.	Retry once, then fall back.	v1 plan
Lessons only need earlier lessons.	Reorder if possible; otherwise retry.	v1 plan
Every question is gradable: a stored correct choice, or accepted answers.	Drop that question; retry if fewer than 2 remain.	v1 plan
Every link is on the allowed list.	Remove the URL, keep the name and search phrase.	v1 plan
Text lengths within limits; no HTML or scripts in text.	Trim or strip.	Built (trimming)
Daily limits per learner.	Build without AI (plans) or show a try-later message (lessons).	Built (plans) / v1 plan (lessons)

AI providers: Groq first (plan model, then the hint model), Cloudflare Workers AI as backup. *Code. Built.* Each saved plan and lesson records the model and the prompt version. *Code. v1 plan.*

11. Quality review

Generated plans and lessons reach learners without being approved first. Review happens **after**, in the Django admin page, and only on a small sample plus anything flagged. It is needed because the code checks in section 10 catch broken structure, not wrong content: a wrong explanation, an example that prints something else, or a quiz whose stored answer is wrong. Beginners cannot spot these. Review is also how the team learns what to fix in the prompts.

Four layers

Layer	What it does	Enforced by	Status
1. Labels and reports	AI-written parts are labelled. Learners can report a problem with their plan (PlanFeedback); v1 plan adds reporting a single lesson.	Code	**Built**
2. Automatic flags	Flag a lesson when: a Python example does not parse (checked with ast; AI or learner code is never run on the server); almost every learner misses the same question (its answer is likely wrong); or it was regenerated many times.	Code	**v1 plan**
3. Lesson feedback	A thumbs up or down at the end of each lesson. Thumbs down flags the lesson.	Code	**v1 plan**
4. Weekly review in admin	A person reviews every flagged or reported lesson plus about 10 random ones, using the checklist below.	Team	**v1 plan**

The review page (Django admin)

- A list of generated lessons (and plan outlines) with filters: **Reported**, **Flagged**, **Thumbs down**, **Not reviewed**, **Prompt version**, and a **Random sample** view.
- Each item has a review status (**Not reviewed**, **OK**, **Problem**), a notes field, who reviewed it and when, and the reason it was flagged.
- Actions: mark OK; mark Problem with a note; **Regenerate for this learner** (the learner sees the new version next time and is not penalised for answers to the old quiz).
- No separate tool or dashboard: the review lives in the admin page curators already use.

The weekly routine

- One person a week, on a rotation, spends about 30 to 60 minutes.
- Review every Reported, Flagged and Thumbs-down item first, then about 10 random lessons.
- When the same problem appears more than once, fix the prompt (it fixes every future lesson) and commit the change with a new prompt version.
- Listed in the README's "Regular jobs" next to check_links and send_reminders, so CompSAt can run it after handover.
- **Problem rate per prompt version** (Problems ÷ reviewed) shows whether a prompt change helped. When it stays low for about four weeks, drop the random sample and review only flagged and reported items.

Review checklist

- Assumptions and any conflict are stated at the top.
- Setup comes first and ends with its check.
- No lesson needs an idea that has not been taught yet.
- Lessons fit the depth table: ideas, key terms, minutes.
- Every lesson has an example, a try-it task, resources and a mini-check.
- Explanations and examples are correct. Run every code example.
- Quiz answers are right, and questions only cover what was taught.
- Resources are on the allowed list and fit the level.
- The project uses what the plan taught and matches the learner's goal.
- Tone is plain and encouraging; English or Taglish matches the learner.

12. What changes in the app (v1 plan)

Area	Change
docs/plan-framework.md	This framework in markdown, kept in git as the source the prompts follow.
learning/tailoring.py	Outline generation replaces topic selection; curated plan stays as the fallback.
New models	PlanModule; GeneratedLesson (lesson JSON, links, model, prompt version, level).
Lesson page	Generate on first open, then show the saved lesson; regenerate after a low score.
Quiz and grading	Grade the lesson's own questions with today's grading rules and score bands.
Review (admin)	Review status, notes, reviewer and flag reason on generated lessons; filters, random sample and a Regenerate action (section 11).
Flags and feedback	Example parse check, most-missed question, regeneration count; thumbs up or down on each lesson.
Finish screen	Glossary from the saved lessons' key terms.
Settings	AI_LESSON_DAILY_LIMIT next to the existing plan and hint limits.
README	Replace "CodeAgapay doesn't write tutorials", update the plan diagram, add the weekly review to Regular jobs.

13. Later

Item	Why it waits
Retrieval: save pages from the allowed sites and have the AI teach from that text, citing it.	Needs page fetching, storage and licence checks (MDN is CC-BY-SA; W3Schools forbids copying).
Lesson library: promote reviewed or well-rated generated lessons so later learners reuse them.	Needs ratings and enough real lessons first. v1 saves lessons in a form that allows it.
Module quizzes, final check, review lessons, catch-up sessions.	Keep v1 small; add once lessons work.
Skip a lesson at 85% before it.	Needs a pre-check per lesson.
AI grading of written answers.	Needs a reliable rubric and costs AI calls per answer.
Spaced repetition instead of the fixed 7-day review.	Known issue; separate change.
Time per session question in onboarding.	Default 30 minutes until the team decides.

14. Team decisions

Decided (9 November 2026)

1. The AI creates the whole learning plan and its lessons, guided by this framework. Pre-approved content blocks are not used in v1.
2. This framework and its v1 plan are accepted as the implementation plan.
3. Generated lessons reach learners **before** anyone reviews them, labelled as AI-written. Review happens afterwards, weekly, in the Django admin page (section 11).
4. The language-learning parts of version 1 stay removed. Only Taglish compatibility is kept (section 6).

Still open

1. Confirm the allowed list in section 9, and who checks each row.
2. Set the review rotation (who reviews which week).
3. Choose the daily lesson-generation limit per learner.
4. Agree what learners see on non-Python tracks when the AI is unavailable.
5. Keep 30 minutes as the default session length, or add the question to onboarding.

Appendix A. Output shapes

Shortened shapes for the two AI steps. Field names follow the app's code style. The full JSON schemas will live in code next to the validation, and this appendix will be updated to match them.

Outline (step 2)

```
{
  "goal": "what the learner can do at the end",
  "level": "beginner | intermediate | advanced",
  "assumptions": ["30-minute sessions (not asked)", "..."],
  "conflict": "empty, or the problem and a realistic version",
  "modules": [{
    "title": "...", "goal": "one sentence", "milestone": "you can now ...",
    "stuck_path": ["reread the key terms", "try the video instead"],
    "lessons": [{
      "key": "m1-l1", "title": "...", "you_will_be_able_to": "...",
      "minutes": 25, "importance": "core | optional",
      "kind": "foundation | recap | core", "needs": ["earlier lesson keys"]
    }]
  }],
  "project": {"title": "...", "description": "...",
              "milestones": [{"lessons": ["m1-l2"], "text": "one step"}],
              "done_when": ["3 to 5 checks"], "idea": 0},
  "next_steps": ["..."]
}
```

Lesson (step 3)

```
{
  "warm_up": [question],
  "key_terms": [{"term": "...", "definition": "..."}],
  "why_it_matters": "...",
  "explanation": "...",
  "examples": [{"code": "...", "output": "...", "note": "..."}],
  "try_it": {"prompt": "...", "starter_code": "...", "sample_answer": "...",
             "checklist": ["..."]},
  "resources": [{"name": "...", "role": "primary | optional",
                 "url": "allowed site, or omitted", "search_phrase": "..."}],
  "mini_check": [question]
}

question = {"format": "multiple_choice | fill_blank | self_check",
            "skill": "recall | apply | explain",
            "prompt": "...", "choices": ["..."], "correct_index": 0,
            "accepted_answers": ["..."], "explanation": "one sentence"}
```

Appendix B. Changes from version 1

v1	v2
Any topic	CodeAgapay’s coding paths and tracks only.
One AI call writes the whole plan	Outline at sign-up; each lesson when first opened.
Rules enforced by the prompt	Each rule says whether the prompt, the code or the team enforces it.
Five topic types	One: tool or technical skill. Setup stays curated.
Any question format	Only auto-gradable formats; “explain” is an ungraded self-check.
Resource dictionary of 40+ general resources, some paid	Allowed list of free coding resources, checked in code.
JSON schema with camelCase fields	Shorter shapes in appendix A, matched to the app.
No implementation status	Every rule marked Built, v1 plan or Later.
Language topic type (spoken languages)	Removed. Only Taglish compatibility of the AI’s own writing.
No review process	Review after learners see content, weekly, in the Django admin page.
—	New: checks on every AI reply, fallback plan, hints-not-answers, team decisions.